

Full Proof Outlines

- A formal proof can be very long and contains repetitive information. A **proof outline** is a way to write out all the information that you would need to generate formal proof system, but with less repetition; so, they're much shorter, and they don't mask the overall structure of the program the way a full proof does.
 - Before studying how to write a **full proof outline**, let us look at an example. Here we give the full proof outline for the program we proved in the previous lecture; I use a different color to show the parts that are added to the program.
1. Give a full proof outline for the program in Lecture 15, Question 3.

```

{ $n \geq 0$ }
 $k := 0$ ; { $n \geq 0 \wedge k = 0$ }  $s := 0$ ; { $n \geq 0 \wedge k = 0 \wedge s = 0$ }
{inv  $p \equiv 0 \leq k \leq n \wedge s = \text{sum}(0, k)$ }
while  $k < n$  do
    { $p \wedge k < n$ }
    { $p[k + 1 / k][s + k + 1 / s]$ }  $s := s + k + 1$ ; { $p[k + 1 / k]$ }  $k := k + 1$  { $p$ }
od
{ $p \wedge k \geq n$ }
{ $s = \text{sum}(0, n)$ }

```

- To get a full proof outline, we annotate statements with their preconditions and postconditions, so that :
 - **Every statement S in the program is wrapped in a triple.**
 - **Every triple $\{p\} S \{q\}$ in the full proof outline is provable.**
 - **Every pair of adjacent conditions $\{p\}\{q\}$ has that $p \Rightarrow q$.**

If you compare the above proof outline with the proof system for the same program, you will find that each judgement from the proof system is shown in the outline.

- Here are the rules to create proof outlines for individual statements:
 - Assignment and **skip** statements that are proved by axiom are handled just as they are in the axiom:
 - $\{p\} v := e \{q\}$
 - $\{p\} \text{skip} \{p\}$
 - Sequence statement that combines $\{p\} S_1 \{q\}$ and $\{q\} S_2 \{r\}$ and getting $\{p\} S_1; S_2 \{r\}$ is written as:
 - $\{p\} S_1; \{q\} S_2 \{r\}$
 - Sequence statement that combines $\{p\} S_1 \{q_1\}$ and $\{p_2\} S_2 \{r\}$ (so $q_1 \Rightarrow p_2$) and getting $\{p\} S_1; S_2 \{r\}$ is written as:
 - $\{p\} S_1; \{q_1\} \{p_2\} S_2 \{r\}$
 - To express the while loop rule that loops through $\{p \wedge B\} S \{p\}$ to get **{inv p }** **while** B **do** S **od** $\{p \wedge \neg B\}$, we write:
 - **{inv p }** **while** B **do** $\{p \wedge B\} S \{p\}$ **od** $\{p \wedge \neg B\}$
 - For conditional rule 1 we write:
 - $\{p\} \text{if } B \text{ then } \{p \wedge B\} S_1 \{q_1\} \text{ else } \{p \wedge \neg B\} S_2 \{q_2\} \text{ fi } \{q_1 \vee q_2\}$

- For conditional rule 2 we write:
 - $\{(B \rightarrow p_1) \wedge (\neg B \rightarrow p_2)\} \text{ if } B \text{ then } \{p_1\} S_1 \{q_1\} \text{ else } \{p_2\} S_2 \{q_2\} \text{ fi } \{q_1 \vee q_2\}$
 - For nondeterministic conditional rule we write:
 - $\{p\} \text{ if } B_1 \rightarrow \{p \wedge B_1\} S_1 \{q_1\} \square B_2 \rightarrow \{p \wedge B_2\} S_2 \{q_2\} \text{ fi } \{q_1 \vee q_2\}$
 - To strengthen the precondition of $\{p\} S \{q\}$ with $p_1 \Rightarrow p$, we write:
 - $\{p_1\} \{p\} S \{q\}$
 - To weaken the postcondition of $\{p\} S \{q\}$ to $q \Rightarrow q_1$, we write:
 - $\{p\} S \{q\} \{q_1\}$
2. Create a full proof outline of the following formal proof.
- | | |
|---|--------------------------|
| 1. $\{T\} k := 0 \{k = 0\}$ | forward assignment |
| 2. $\{k = 0\} x := 1 \{x = 1 \wedge k = 0\}$ | forward assignment |
| 3. $\{T\} k := 0; x := 1 \{x = 1 \wedge k = 0\}$ | sequence 1,2 |
| 4. $x = 1 \wedge k = 0 \rightarrow k \geq 0 \wedge x = 2^k$ | predicate logic |
| 5. $\{T\} k := 0; x := 1 \{k \geq 0 \wedge x = 2^k\}$ | weaken postcondition 3,4 |
- We can create a full proof outline following the proof: there is assignment, assignment, sequence and weaken postcondition; so, we can create:

$$\{T\} k := 0; \{k = 0\} x := 1 \{x = 1 \wedge k = 0\} \{k \geq 0 \wedge x = 2^k\}$$
3. Create formal proof for the following full proof outline:
- $$\{T\} \{0 \geq 0 \wedge 1 = 2^0\} k := 0; \{k \geq 0 \wedge 1 = 2^k\} x := 1 \{k \geq 0 \wedge x = 2^k\}$$
- There are two statements, and they are in sequence.

1. $\{k \geq 0 \wedge 1 = 2^k\} x := 1 \{k \geq 0 \wedge x = 2^k\}$	backward assignment
2. $\{0 \geq 0 \wedge 1 = 2^0\} k := 0 \{k \geq 0 \wedge 1 = 2^k\}$	backward assignment
3. $\{0 \geq 0 \wedge 1 = 2^0\} k := 0; x := 1 \{k \geq 0 \wedge x = 2^k\}$	sequence 2,1
4. $T \rightarrow 0 \geq 0 \wedge 1 = 2^0$	predicate logic
5. $\{T\} k := 0; x := 1 \{k \geq 0 \wedge x = 2^k\}$	strengthen precondition 4,3
- Both Examples 2 and 3 are proving the triple $\{T\} k := 0; x := 1 \{k \geq 0 \wedge x = 2^k\}$, but we have different formal proofs and different proof outlines. The reason here is that the forward assignment and the backward assignment are equally powerful.
In general, a program can be proved in different ways. **For each formal proof, there is a corresponding full proof outline.**

Minimal Proof Outlines and Partial Proof Outlines

- In the previous class, we learned how to get a full proof outline from a formal proof: in a program, we wrap each statement with the triple used in the formal proof for this program.
 - For the sake of completeness, we do not have a good way to show conjunction/disjunction rule in a proof outline, since we need to wrap the same statement in different triples.
- In a **minimal proof outline**, we have the minimum amount of program annotation that allows us to infer the rest of the formal proof outline: in general, we can't infer the *initial precondition* and *initial postcondition*, nor can we infer the *invariants of loops*, so a minimal outline will include those conditions and no others.

- A **partial proof outline** is somewhere in the middle: More filled-in than a minimal outline but not completely full.

4. In Question 1, we have seen the following full proof outline:

```

{n ≥ 0}
k := 0; {n ≥ 0 ∧ k = 0} s := 0; {n ≥ 0 ∧ k = 0 ∧ s = 0}
{inv p ≡ 0 ≤ k ≤ n ∧ s = sum(0, k)}
while k < n do
    {p ∧ k < n}
    {p[k + 1 / k][s + k + 1 / s]} s := s + k + 1; {p[k + 1 / k]} k := k + 1 {p}
od
{p ∧ k ≥ n}
{s = sum(0, n)}

```

What is the minimal proof outline of the above statement? (Use red color)

5. Consider the following full proof outlines and their minimal proof outlines:

- $\{T\} k := 0; \{k = 0\} x := 1 \{x = 1 \wedge k = 0\} \{k \geq 0 \wedge x = 2^k\}$
 - $\{T\} \{0 \geq 0 \wedge 1 = 2^0\} k := 0; \{k \geq 0 \wedge 1 = 2^k\} x := 1 \{k \geq 0 \wedge x = 2^k\}$
- o These full proof outlines have the same minimal proof outlines: $\{T\} k := 0 \ x := 1 \{k \geq 0 \wedge x = 2^k\}$. The reason multiple full proof outlines can have the same minimal outline is because different organizations of backward assignment and forward assignment can have the same minimal outline. There can also be differences in whether and where preconditions are strengthened or postconditions are weakened.

Expanding Minimal/Partial Proof Outlines to Full Proof Outlines

- Remember that, in a full proof outline, every statement needs to be wrapped in a probable triple. To expand a minimal/partial proof outline into a full proof outline, basically we need to infer all the missing predicates. Postconditions are inferred from preconditions using $sp(\dots)$, and preconditions are inferred from postconditions using $wlp(\dots)$. Loop invariants tell us how to annotate the loop body and postcondition.
 - Expanding a minimal/partial outline can lead to several different full outlines (because multiple full outlines can have the same minimal outline), but all the full outlines will be correct, and the differences between them are generally stylistic.
6. Expand the following minimal proof outline to a full proof outline. Here $abs(x)$ gives the absolute value of x .
- $$\{y = x\} \text{ if } x < 0 \text{ then } y := -x \text{ fi } \{y = abs(x)\}$$

- First, let's use wlp on each branch then use Conditional Rule 1 for the whole conditional statement. Then we have:

```

{y = x}
if x < 0 then
    {y = x ∧ x < 0} {−x = abs(x)} y := −x {y = abs(x)}
else
    {y = x ∧ x ≥ 0} {y = abs(x)} skip {y = abs(x)}
fi {y = abs(x) ∨ y = abs(x)}
{y = abs(x)}

```

- b. Let's use *sp* on each branch then use Conditional Rule 1 for the whole conditional statement. Then we have:

$$\begin{aligned}
 & \{y = x\} \\
 & \mathbf{if} \ x < 0 \ \mathbf{then} \\
 & \quad \{y = x \wedge x < 0\} \ y := -x \ \{y_0 = x \wedge x < 0 \wedge y = -x\} \ \{y = \mathit{abs} \ (x)\} \\
 & \mathbf{else} \\
 & \quad \{y = x \wedge x \geq 0\} \ \mathbf{skip} \ \{y = x \wedge x \geq 0\} \ \{y = \mathit{abs} \ (x)\} \\
 & \mathbf{fi} \ \{y = \mathit{abs} \ (x)\}
 \end{aligned}$$

- c. Let's use *wlp* on each branch then use Conditional Rule 2 for the whole conditional statement. Then we have:

$$\begin{aligned}
 & \{y = x\} \\
 & \{(x < 0 \rightarrow -x = \mathit{abs} \ (x)) \wedge (x \geq 0 \rightarrow y = \mathit{abs} \ (x))\} \\
 & \mathbf{if} \ x < 0 \ \mathbf{then} \\
 & \quad \{-x = \mathit{abs} \ (x)\} \ y := -x \ \{y = \mathit{abs} \ (x)\} \\
 & \mathbf{else} \\
 & \quad \{y = \mathit{abs} \ (x)\} \ \mathbf{skip} \ \{y = \mathit{abs} \ (x)\} \\
 & \mathbf{fi} \ \{y = \mathit{abs} \ (x)\}
 \end{aligned}$$